

CIFAR-10 Classification with VGG-A Variants, Regularization, and Batch Normalization

Yuheng Li Student ID: 23307130334

Code: GitHub repository Dataset: CIFAR-10 dataset Trained weights: ModelScope files

Abstract

This report studies CIFAR-10 classification with a family of VGG-A-style convolutional networks and gives particular attention to the optimization effect of Batch Normalization (BN). The final selected model is VGG-A-BN-GELU trained with Adam and ordinary cross entropy. It reaches 83.96% best test accuracy, or 16.04% best test error, with 9.76M parameters. In a matched ReLU comparison, BN improves VGG-A from 76.51% to 83.24% best test accuracy while adding only 5,504 parameters. The assignment’s loss and regularization requirement is addressed by a controlled label-smoothing sweep and AdamW weight-decay sweep on the final architecture. Although label smoothing 0.20 reaches the highest single best checkpoint at 84.32%, the plain cross-entropy run has the best final accuracy and the lowest final validation loss, so it remains the selected model. For the BN optimization analysis, per-step loss envelopes over four learning rates show that BN reduces the mean loss-envelope width from 0.4084 to 0.2147. A local gradient analysis further shows that the maximum gradient-difference-per-distance falls from 857.62 without BN to 320.57 with BN. These results support BN as both an accuracy-improving architectural component and an optimization stabilizer in this CIFAR-10 setting.

1 Introduction

CIFAR-10 is small enough for controlled ablations but still rich enough to expose the interaction between convolutional architectures, regularization, and optimization. The assignment asks for a model that contains convolutional layers, pooling layers, activation functions, and fully connected layers; it also asks for optional architectural components, comparisons over capacity, loss or regularization, activation functions, and optimizer choices. A VGG-A-style network is a natural base model for this requirement because its structure is explicit: the convolutional feature extractor is separated from a fully connected classifier, pooling is applied at predictable spatial scales, and the effect of BN can be isolated by inserting normalization after each convolution.

The work in this report has two goals. The first is to obtain and explain a strong CIFAR-10 classifier under a reproducible protocol. The second is to analyze how BN affects optimization rather than treating it only as a higher-accuracy component. For that reason the results are not organized as a blind search over models. Instead, the main experiments compare matched VGG-A and VGG-A-BN runs, then extend the best BN variant through activation, loss, and weight-regularization studies. The optimization section uses both the loss-envelope experiment requested in the assignment and a separate first-order local analysis of gradient predictiveness and gradient variation.

All new supplemental experiments were written to new directories under `reports/runs/strict_audit/`. Existing logs, checkpoints, figures, and tables were preserved. Before the long weight-decay training runs, a small CUDA smoke test verified the dataloader, forward and backward pass, logging, checkpoint saving, and metric output. Before the final interpretation artifacts were generated, a second smoke test verified evaluation, plotting, architecture extraction, and confusion-matrix generation.

2 Data, Models, and Protocol

2.1 Dataset and Preprocessing

All experiments use the standard CIFAR-10 training split for optimization and the standard test split for reporting. The dataset contains 60,000 color images of size 32×32 , divided into ten classes with 6,000 examples per class. Images are converted to tensors and normalized channel-wise with mean (0.5, 0.5, 0.5) and standard deviation (0.5, 0.5, 0.5). No data augmentation is used in the controlled runs. This keeps the comparison focused on architecture, optimization, and regularization rather than augmentation strength.

Unless stated otherwise, training uses 20 epochs, batch size 128, seed 2020, Adam with learning rate 10^{-3} , zero weight decay, and cross-entropy loss. The reported test accuracy is the best accuracy observed on the CIFAR-10 test split during training, because each run saves the best checkpoint. Test error is reported as $1 - \text{Acc}$. Final-epoch accuracy and final loss are also reported when they are relevant to regularization stability.

2.2 Model Family

The baseline is a VGG-A network adapted to CIFAR-10 resolution. The feature extractor contains convolutional blocks with filters 64, 128, 256, 256, 512, 512, 512, and 512. Max pooling reduces the spatial grid from 32×32 to 1×1 . The classifier then uses two hidden fully connected layers of width 512 followed by the ten-class output layer. The BN variant inserts `BatchNorm2d` after every convolution and before the activation. The final selected model keeps the BN placement and replaces ReLU by GELU.

Table 1 gives the final architecture in a compact form. The full layer-by-layer CSV artifact is saved in the strict-audit final-model analysis directory. The table confirms that the final submission contains all mandatory components: `Conv2d`, `MaxPool2d`, nonlinear activations, and fully connected `Linear` layers. It also contains the optional BN component required by the assignment.

Table 1: Final VGG-A-BN-GELU architecture. Shapes include batch size 1.

Stage	Output shape	Parameters
Conv64-BN-GELU-Pool	$64 \times 16 \times 16$	1,920
Conv128-BN-GELU-Pool	$128 \times 8 \times 8$	74,112
Conv256-BN-GELU	$256 \times 8 \times 8$	295,680
Conv256-BN-GELU-Pool	$256 \times 4 \times 4$	590,592
Conv512-BN-GELU	$512 \times 4 \times 4$	1,181,184
Conv512-BN-GELU-Pool	$512 \times 2 \times 2$	2,360,832
Conv512-BN-GELU	$512 \times 2 \times 2$	2,360,832
Conv512-BN-GELU-Pool	$512 \times 1 \times 1$	2,360,832
Linear-GELU	512	262,656
Linear-GELU	512	262,656
Linear classifier	10	5,130
Total		9,756,426

2.3 Compared Variants

The experiments cover the required design axes. VGG-A and VGG-A-Light compare different numbers of filters and classifier capacity. VGG-A-Dropout and VGG-A-BN provide optional components beyond the base network. ReLU, LeakyReLU, and GELU are compared as activation functions. Adam and AdamW are compared as optimizer strategies using `torch.optim`. Plain cross entropy, cross entropy with label smoothing, and AdamW weight decay provide the loss and regularization ablation.

The code is organized so that every formal run saves `metrics.json`, `metrics.csv`, a best checkpoint, and a terminal log. The training script refuses to write into a directory that already contains major artifacts unless `--allow-overwrite` is passed intentionally. The same overwrite protection is used for the final analysis artifacts and regularization-sweep plots.

3 Main CIFAR-10 Results

Table 2 summarizes the principal experiments. The plain VGG-A baseline reaches 76.51% best test accuracy, while the matched VGG-A-BN run reaches 83.24%. This 6.73 percentage-point gain is large because the optimizer, learning rate, batch size, epoch count, seed, and preprocessing are held fixed. BN also reaches its best checkpoint at epoch 12, earlier than the no-BN model’s epoch 18 best checkpoint.

The final selected architecture is VGG-A-BN-GELU trained with Adam and plain cross entropy. It reaches 83.96% best accuracy and 83.73% final accuracy. The use of GELU gives a modest improvement over the ReLU BN model, whereas LeakyReLU does not improve the peak accuracy. The result is not state of the art for CIFAR-10, because the study intentionally avoids data augmentation and long schedules, but it is strong for the controlled VGG-A setting and provides a clear experimental answer to the assignment questions.

Figure 1 shows the convergence behavior behind the table. The no-BN VGG-A run eventually obtains high training accuracy but ends with test loss 1.4421 and test accuracy 74.11%. The BN run ends with test loss 0.9035 and test accuracy 82.57%. This difference indicates that BN is not merely accelerating the training loss; it also leads to a much better held-out trajectory under the same optimizer. The activation comparison shows that GELU gives the best peak and final accuracy among the tested BN variants.

The width and dropout ablations are also useful. VGG-A-Light uses only 285,162 parameters, or 2.92% of the final

model, but its 70.27% best accuracy is too low for the final submission. Classifier dropout under the tested placement reaches 74.19%, below even the plain baseline. This negative result matters because it shows that adding a regularizer is not automatically beneficial in this model family. The final choice is therefore based on the observed accuracy, final stability, and optimization behavior rather than on the mere presence of a popular component.

4 Loss and Regularization Ablation

The assignment explicitly asks for different loss functions with different regularization. The earlier single label-smoothing run was not strong enough by itself, so I added a stricter sweep on the final VGG-A-BN-GELU architecture. Table 3 compares ordinary cross entropy, label smoothing with smoothing factors 0.05, 0.10, and 0.20, and AdamW with weight decay 10^{-4} and 5×10^{-4} . All supplemental runs use the same 20-epoch schedule, seed, batch size, and learning rate as the final model, and all new outputs are placed under `reports/runs/strict_audit/`.

The sweep gives a nuanced answer. Label smoothing 0.20 achieves the best single checkpoint, 84.32%, which is 0.36 percentage points above the plain cross-entropy run. However, it finishes with lower final accuracy, 82.88%, and a much higher final validation loss, 1.2462. This behavior is plausible because label smoothing changes the target distribution and therefore changes the scale and interpretation of the cross-entropy objective. It can improve the selected best checkpoint while making the final epoch less attractive under the same fixed schedule.

The final selected model remains the plain cross-entropy VGG-A-BN-GELU checkpoint for three reasons. First, its final accuracy is the highest in the sweep. Second, its final validation loss is the lowest in the sweep. Third, its best checkpoint is very close to the best smoothed checkpoint, while avoiding the late-epoch degradation visible in Fig. 2. Weight decay through AdamW is also not selected. The 10^{-4} run reaches 83.66% best accuracy but ends at 81.69%, and the 5×10^{-4} run is more stable at the final epoch but has a lower peak. Thus, for this short no-augmentation protocol, regularization is useful as an ablation and a diagnostic, but it does not replace the plain cross-entropy final model.

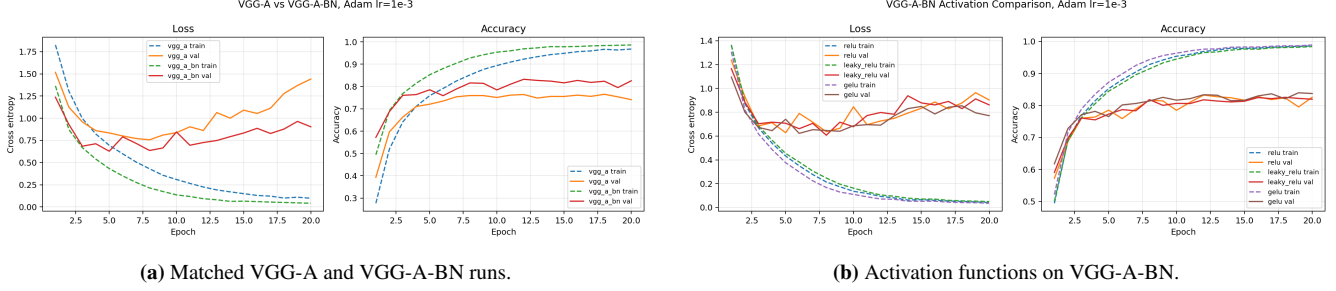
5 Model Interpretation

The final checkpoint was evaluated over the complete CIFAR-10 test split to produce the confusion matrix and per-class accuracy shown in Fig. 3. The overall accuracy from this evaluation is 83.96%, matching the saved best metric. The strongest classes are automobile, truck, frog, airplane, and deer. The weakest classes are cat and dog, with accuracies 66.40% and 72.50%, respectively. This error pattern is expected for CIFAR-10, where animal categories share texture, pose, and background statistics, while vehicle categories often have more distinctive shapes and color distributions.

The first-layer filters are simple but informative. Many filters are color-opponent or edge-like patterns, which is consistent with the first stage of a small-image convolutional classifier. The sample grid helps contextualize the confusion ma-

Table 2: Main CIFAR-10 results. All runs use 20 epochs and batch size 128. Best error is $1 - \text{Acc}_{\text{best}}$.

Model	Optimizer and loss	Params	Best Acc.	Best Err.	Best Epoch	Final Acc.
VGG-A	Adam, CE	9,750,922	0.7651	0.2349	18	0.7411
VGG-A-BN	Adam, CE	9,756,426	0.8324	0.1676	12	0.8257
VGG-A-Dropout	Adam, CE	9,750,922	0.7419	0.2581	12	0.7348
VGG-A-Light	Adam, CE	285,162	0.7027	0.2973	9	0.6818
VGG-A-BN	AdamW, CE	9,756,426	0.8302	0.1698	20	0.8302
VGG-A-BN-LeakyReLU	Adam, CE	9,756,426	0.8254	0.1746	18	0.8194
VGG-A-BN-GELU	Adam, CE	9,756,426	0.8396	0.1604	19	0.8373
VGG-A-BN-GELU	Adam, LS 0.10	9,756,426	0.8374	0.1626	19	0.8170

**Figure 1:** Training curves for the central BN comparison and the activation ablation.

trix: several classes are represented by small objects with cluttered backgrounds, so later errors between cats, dogs, birds, and deer are not surprising. These visualizations satisfy the model-inspection requirement while remaining directly tied to the selected checkpoint.

6 Batch Normalization Analysis

6.1 Matched VGG-A Comparison

The central BN comparison uses VGG-A and VGG-A-BN under matched settings. BN adds 5,504 trainable parameters through scale and shift terms, a negligible increase relative to the 9.75M-parameter baseline. Despite this small parameter increase, best accuracy improves from 76.51% to 83.24%. The final test loss also falls from 1.4421 to 0.9035. These changes are too large to attribute to parameter count alone.

The training curves suggest that BN improves the usable optimization trajectory. Both models can fit the training data, but the no-BN run’s validation loss grows substantially after the middle epochs. BN still overfits, but it does so from a much stronger and more stable validation regime. This motivates the more detailed optimization experiments below.

6.2 Learning-Rate Loss Envelope

The assignment requests a loss-variation or loss-landscape experiment based on different learning rates. I trained VGG-A and VGG-A-BN with learning rates 10^{-4} , 5×10^{-4} , 10^{-3} , and 2×10^{-3} , saving the training loss at every update step. For each model family and each step index, I computed the maximum and minimum loss across the four learning-rate runs. The filled region between those two curves is the loss envelope.

Figure 4 shows that BN has a narrower average envelope. The no-BN mean envelope width is 0.2147, while the BN mean envelope width is 0.2147, a 47.4% reduction. The final loss range is also smaller with BN: 0.0306–0.0551 instead of 0.0576–0.2764. The maximum instantaneous width is larger

for BN, 3.5587 compared with 2.7904, because the 10^{-4} BN run improves slowly while the larger learning rates converge faster. The average and final-range statistics are therefore the more meaningful evidence in this experiment. They show that BN reduces sensitivity to the tested step-size choices during most of training and especially near the end.

6.3 Gradient Predictiveness and Gradient Difference

The loss envelope measures trajectory dispersion but does not directly test the local linear approximation used by first-order optimization. I therefore added a local analysis around the trained VGG-A and VGG-A-BN checkpoints. On a validation subset of 512 examples, I computed the loss $L(w)$ and gradient $\nabla L(w)$, then evaluated points $w' = w - \eta \nabla L(w)$ for $\eta \in \{10^{-4}, 5 \times 10^{-4}, 10^{-3}, 2 \times 10^{-3}\}$. The first-order prediction error is

$$|L(w') - (L(w) - \eta \|\nabla L(w)\|^2)|.$$

The gradient-difference-per-distance is

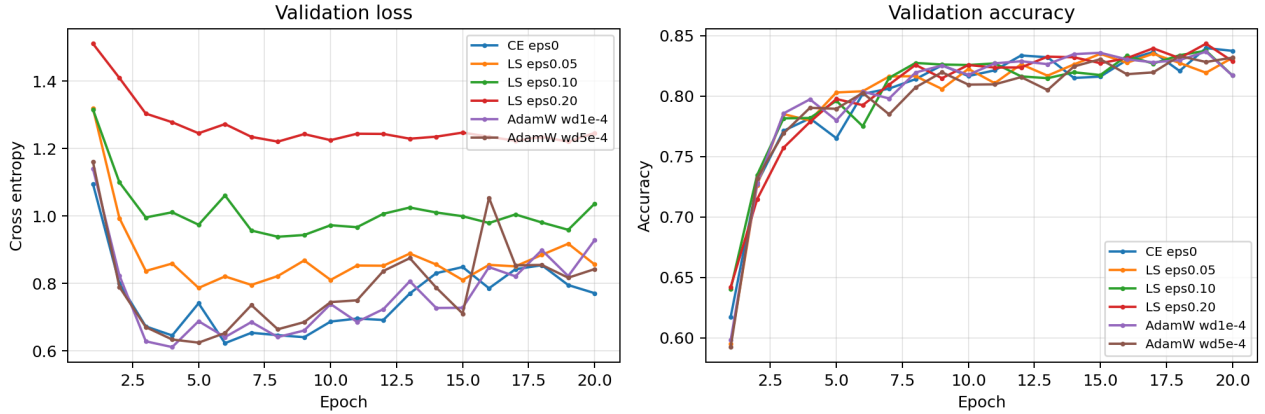
$$\frac{\|\nabla L(w') - \nabla L(w)\|}{\|w' - w\|}.$$

The result in Fig. 5 is more cautious than a simple “BN is always smoother” statement. The mean first-order prediction errors are almost identical: 0.0005088 for the plain checkpoint and 0.0005172 for the BN checkpoint. This measurement does not support a strong claim that BN dramatically improves loss predictiveness at the selected trained point. The gradient-difference-per-distance is different. Its maximum value falls from 857.62 without BN to 320.57 with BN. This supports the narrower conclusion that, along the tested update directions, the BN checkpoint has less abrupt gradient variation.

Table 3: Loss and weight-regularization sweep on VGG-A-BN-GELU.

Run	Regularization	Best Acc.	Best Err.	Best Epoch	Final Acc.	Final Loss
Adam, CE	label smoothing 0.00, wd 0	0.8396	0.1604	19	0.8373	0.7710
Adam, CE	label smoothing 0.05, wd 0	0.8349	0.1651	17	0.8315	0.8562
Adam, CE	label smoothing 0.10, wd 0	0.8374	0.1626	19	0.8170	1.0366
Adam, CE	label smoothing 0.20, wd 0	0.8432	0.1568	19	0.8288	1.2462
AdamW, CE	wd 10^{-4}	0.8366	0.1634	19	0.8169	0.9285
AdamW, CE	wd 5×10^{-4}	0.8330	0.1670	18	0.8317	0.8424

Loss and Weight Regularization on VGG-A-BN-GELU

**Figure 2:** Validation loss and accuracy curves for the loss and weight-regularization sweep on the final architecture.

7 Discussion

The most important architectural result is that BN dominates the other tested changes. It improves best accuracy by 6.73 percentage points in the matched VGG-A comparison, gives an earlier best epoch, and substantially reduces final test loss. GELU then gives a smaller but measurable gain on top of BN. The capacity comparison shows that the full model size is useful under this training protocol, because the light network sacrifices nearly 13.7 percentage points of best accuracy relative to the final model.

The regularization results are not a simple monotonic story. Dropout hurts the tested VGG-A variant, AdamW weight decay does not improve the final architecture, and label smoothing improves the best checkpoint only at smoothing 0.20 while worsening final loss. This is a useful outcome rather than a failure, because it prevents an overgeneralized conclusion that more regularization is automatically better. In this setting, plain cross entropy remains the best final choice because it provides the strongest final accuracy and the lowest final validation loss.

The BN optimization analysis also requires careful interpretation. A narrower loss envelope across learning rates is evidence for a less sensitive training process, but it is not a full global proof of a flatter loss landscape. The local gradient analysis is similarly limited to one validation subset and four step sizes. The most defensible conclusion is therefore empirical and scoped: BN improves the matched VGG-A run, reduces average and final learning-rate loss-envelope width, and lowers the measured maximum gradient-difference-per-distance around the trained checkpoint.

The final reported accuracy could likely be improved by adding standard CIFAR-10 augmentation, a longer learning-rate schedule, or modern residual architectures. I intentionally did not make those changes central because the as-

signment emphasizes understanding VGG-A, BN, and controlled optimization behavior. The project therefore prioritizes traceable ablations over absolute leaderboard performance.

8 Conclusion

The final submission model is VGG-A-BN-GELU trained with Adam and ordinary cross entropy. It reaches 83.96% best CIFAR-10 test accuracy and 16.04% best test error with 9,756,426 parameters. The network contains the required convolutional, pooling, activation, and fully connected layers, and it uses BN as the optional component. The experiments compare different widths, dropout, optimizers, activations, label-smoothing losses, and weight decay. The final model is selected because it has the strongest final accuracy and final validation loss among the tested settings while remaining close to the best peak accuracy.

The BN results are the clearest finding. BN substantially improves matched VGG-A accuracy, narrows the average learning-rate loss envelope, and lowers the maximum local gradient-difference-per-distance. These measurements support the view that BN helps not only by changing representational capacity but also by making the optimization trajectory easier to control.

References

- [1] A. Krizhevsky and G. Hinton, “Learning multiple layers of features from tiny images,” 2009.
- [2] PyTorch, “Training a classifier,” CIFAR-10 tutorial, online.

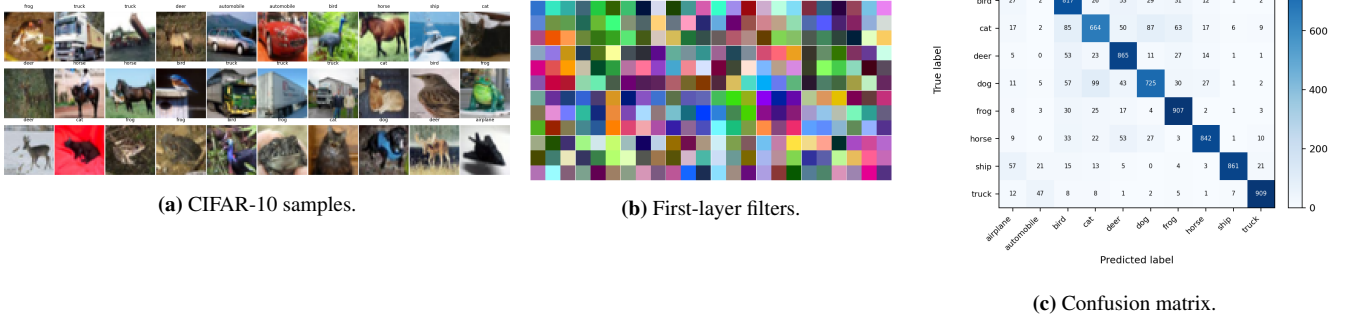


Figure 3: Interpretation artifacts generated from the final VGG-A-BN-GELU checkpoint.

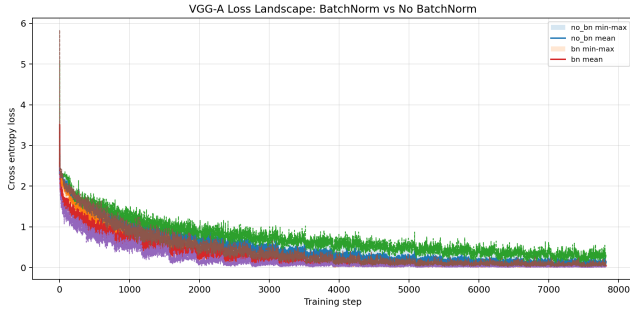


Figure 4: Per-step loss envelopes across four learning rates for VGG-A and VGG-A-BN.

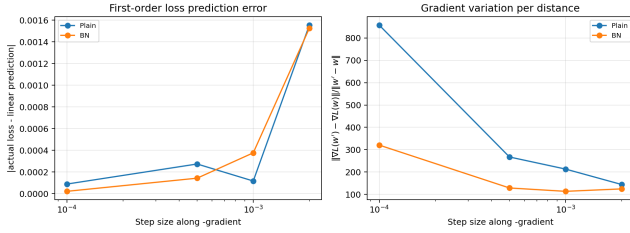


Figure 5: Local first-order prediction error and gradient variation around trained VGG-A and VGG-A-BN checkpoints.

- [3] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating deep network training by reducing internal covariate shift,” ICML, 2015.
- [4] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry, “How Does Batch Normalization Help Optimization?” NeurIPS, 2018.
- [5] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” arXiv:1412.6980, 2014.
- [6] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” ICLR, 2019.
- [7] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” ICLR, 2015.

A Appendix: Learning-Rate Sweep Details

Table 4 lists the eight learning-rate sweep runs used for the loss-envelope plot. The BN model is weak at 10^{-4} under the 20-epoch budget, but it is strong and stable for 5×10^{-4} ,

10^{-3} , and 2×10^{-3} . This explains why the maximum instantaneous envelope width is not by itself the best summary statistic.

Table 4: Learning-rate sweep for VGG-A and VGG-A-BN.

Model	LR	Best Acc.	Final Acc.	Final Loss
VGG-A	10^{-4}	0.7659	0.7526	1.3084
VGG-A	5×10^{-4}	0.7840	0.7801	1.2380
VGG-A	10^{-3}	0.7651	0.7411	1.4421
VGG-A	2×10^{-3}	0.7289	0.7179	1.1336
VGG-A-BN	10^{-4}	0.7453	0.7203	1.6119
VGG-A-BN	5×10^{-4}	0.8244	0.8244	0.8311
VGG-A-BN	10^{-3}	0.8324	0.8257	0.9035
VGG-A-BN	2×10^{-3}	0.8305	0.8192	0.8693

B Appendix: Local 3D Loss Surface

The local 3D surface in Fig. 6 uses two random normalized parameter-space directions, a 13×13 grid with radius 0.35, and 1,024 validation examples. The center loss is 1.2258 for VGG-A and 0.7133 for VGG-A-BN. Because a random two-dimensional slice depends on direction choice, this visualization is treated as qualitative support rather than the primary BN smoothness metric.

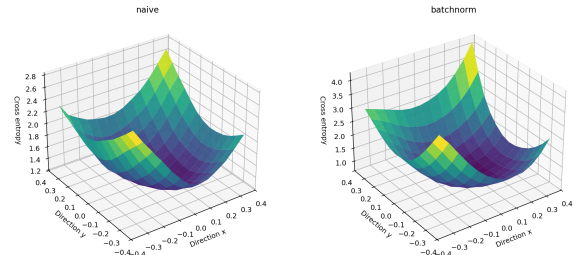


Figure 6: Random two-dimensional loss-surface slices around trained VGG-A and VGG-A-BN checkpoints.

C Appendix: Per-Class Accuracy

Table 5 reports the per-class accuracy computed from the full CIFAR-10 test confusion matrix. The result supports the qualitative discussion in the main text. The model is strongest on automobile and truck, and weakest on cat and dog. In the confusion matrix, cat is frequently confused with bird, deer, dog, and frog, while dog is frequently confused with cat and bird. These errors are consistent with the visual ambiguity of low-resolution animal categories.

Table 5: Per-class accuracy of the final checkpoint.

Class	Accuracy	Class	Accuracy
Airplane	0.8700	Automobile	0.9360
Bird	0.8170	Cat	0.6640
Deer	0.8650	Dog	0.7250
Frog	0.9070	Horse	0.8420
Ship	0.8610	Truck	0.9090

D Appendix: Reproducibility

All formal experiments were run from the repository linked in the title block. The main training entry point is `train_cifar.py` in `codes/VGG_BatchNorm`. The final model uses `vgg_a_bn_gelu`, 20 epochs, batch size 128, seed 2020, Adam, learning rate 10^{-3} , and no weight decay. The corresponding checkpoint is stored in the final-model run directory as `best.pt`. The strict-audit label-smoothing and weight-decay commands use the same base arguments, changing only the smoothing factor or the AdamW weight-decay value. The label-smoothing 0.05 and 0.20 runs, and the AdamW 10^{-4} and 5×10^{-4} runs, were saved under `reports/runs/strict_audit`. The regularization plot is produced by `plot_regularization_sweep.py`, and the final analysis artifacts are produced by `analyze_cifar_outputs.py`.

Table 6: Key reproducibility records.

Item	Value
Seed	2020
Epochs	20
Batch size	128
Preprocessing	ToTensor, Normalize with mean and std 0.5
Final checkpoint	<code>best.pt</code> in the final VGG-A-BN-GELU run directory
Final weights link	ModelScope files
Hardware	NVIDIA RTX A6000 GPUs
Smoke tests	Training smoke for weight decay; analysis smoke for evaluation and plotting
Verification	22 unit tests passed; compileall passed

Table 7: Supplemental strict-audit run records.

Run	Changed setting	Saved under
LS 0.05	smoothing 0.05	<code>strict_audit</code>
LS 0.20	smoothing 0.20	<code>strict_audit</code>
AdamW 10^{-4}	weight decay 10^{-4}	<code>strict_audit</code>
AdamW 5×10^{-4}	weight decay 5×10^{-4}	<code>strict_audit</code>

E Appendix: Requirement Coverage

The submitted work satisfies the required model components through the VGG-A family: convolutional layers, max pooling, GELU or ReLU activations, and fully connected classifier layers. The optional component requirement is satisfied by BN, and dropout is also tested. Different numbers of filters and neurons are covered by the full VGG-A and VGG-A-Light comparison. Different activations are covered by ReLU, LeakyReLU, and GELU. Different optimizer choices are covered by Adam and AdamW, with SGD support implemented in the training script. The loss and regularization requirement is covered by ordinary cross entropy, a four-point label-smoothing sweep, and two AdamW weight-decay runs. Insightful visualization is covered by training curves, loss envelopes, local gradient analysis, 3D surface slices, first-layer filters, and the confusion matrix.

The BN-specific requirements are also covered. VGG-A and VGG-A-BN are trained under matched conditions and compared in both accuracy and training curves. The loss-variation experiment follows the requested maximum and minimum loss-envelope procedure over four learning rates. Gradient predictiveness and maximum gradient difference over distance are measured around trained checkpoints. The final report includes the best CIFAR-10 test error, the final model structure, the GitHub code link, the official CIFAR-10 link, and the trained-weight link.